

Дополнительная лекция по Си № 10

21 мая 2025 г.

Коновалов А.В.

Функции с переменным числом параметров (продолжение)

Передача списка параметров в другие функции

Список параметров `va_list` можно передавать в другие функции и в них его анализировать. Чаще всего эта особенность используется для написания собственных обёрток над функциями форматированного ввода и вывода.

Функции `vprintf` и `vscanf` вместо дополнительных параметров принимают `va_list`.

В качестве примера напомним функцию `my_log(fmt, ...)`, которая печатает текущее время и пользовательское сообщение.

Примечание. Сначала мы хотели назвать функцию `log(fmt, ...)`, но получили такое сообщение ошибки от GCC:

```
D:\Си>gcc -Wall -Wextra log-example.c
log-example.c:7:6: warning: conflicting types for built-in function 'log' [-Wbuiltin-declaration-mismatch]
void log(const char *fmt, ...)
```

Оказывается, функция, вычисляющая логарифм, не объявляется в файле `math.h`, а является встроенной из соображений оптимизации (в системе команд x86 есть подмножество команд арифметического сопроцессора, включающее аппаратную реализацию логарифма, оптимизация, вероятно, для этого).

Поэтому нам пришлось назвать функцию немного иначе: `my_log`. *Конец примечания.*

Рассмотрим реализацию функции `my_log`:

```
#include <stdarg.h>
#include <stdio.h>
#include <string.h>
#include <time.h>
```

```

void my_log(const char *fmt, ... )
{
    time_t now;
    char str_now[30], *pn;

    time(&now);
    strncpy(str_now, ctime(&now), sizeof(str_now));
    str_now[sizeof(str_now) - 1] = '\0';
    pn = strchr(str_now, '\n');
    if (pn) *pn = '\0';
    printf("[%s] ", str_now);

    va_list args;
    va_start(args, fmt);
    vprintf(fmt, args);
    va_end(args);
}

int main()
{
    my_log("7 * 7 = %d, hello, %s!\n", 7 * 7, "student");
    return 0;
}

```

Следующий фрагмент кода позволяет получить и распечатать строковое представление временного штампа:

```

time_t now;
char str_now[30], *pn;

time(&now);
strncpy(str_now, ctime(&now), sizeof(str_now));
str_now[sizeof(str_now) - 1] = '\0';
pn = strchr(str_now, '\n');
if (pn) *pn = '\0';
printf("[%s] ", str_now);

```

Функция `time` позволяет получить значение текущего времени в виде значения типа `time_t` (согласно Стандарту, это синоним одного встроенных числовых типов, конкретный тип зависит от реализации, в частности, он может быть либо целым, либо вещественным). Функция `ctime` возвращает указатель на строку, содержащую текстовое представление времени, переданного как параметр.

Возвращаемая строка всегда завершается на знак перевода строки, который нам не нужен. Поэтому мы копируем строку в буфер `str_now`, содержимое которого «обрезаем» по первому символу `\n`, присваивая в данную позицию концевой ноль `\0`. Менять непосредственно строку, которая возвращается `ctime`, мы не будем,

т.к. эту строку контролируем не мы, а библиотека (копию контролируем мы сами). Дополнительные действия с буфером демонстрируют работу со строкой, длину которой мы не знаем: копируем её в буфер некоторого размера при помощи `strncpy`, на случай более длинной строки в конец принудительно добавляем `\0` (т.к. функция `strncpy` ноль может не добавить).

Затем просто печатаем временной штамп внутри квадратных скобок.

Стандартная библиотека языка Си позволяет точнее управлять форматированием строчек — см. документацию к функции `strftime()` (и вообще, к файлу `<time.h>`).

Время, возвращаемое функцией `time` в виде переменной типа `time_t` на POSIX-системах отсчитывается в секундах от полночи 1 января 1970 года — т.н. «Эра UNIX». При использовании `int32_t` (32-битного целого числа) для представления `time_t` в 2038 году произойдёт переполнение — время будет считаться отрицательным и означать какое-то число начала 20 века. Это со страхом ожидаемое событие называют Проблемой 2038, предполагается, что какие-то программы могут перестать работать в этот момент.

Для нас наиболее важны следующие строчки:

```
va_list args;
va_start(args, fmt);
vprintf(fmt, args);
va_end(args);
```

Здесь мы инициализируем `va_list args`, которая ссылается на параметры после параметра `fmt` и передаём её в функцию `vprintf`, которая получает форматную строку и `va_list` и выполняет форматирование как обычная `printf`. Затем деинициализируем `args` при помощи `va_end` (бывают платформы, где это необходимо). Для всех функций семейства `...printf` и `...scanf` есть двойники, принимающие `va_list` и их имена начинаются на `v`: `vprintf`, `vfprintf`, `vsprintf`, `vscanf`, `vfscanf`, `fsscanf`, `vsnprintf`.

Пример работы функции `log_ex`:

```
D:\Си>gcc -Wall -Wextra log-example.c
```

```
D:\Си>a.exe
```

```
[Wed May 21 15:17:09 2025] 7 * 7 = 49, hello, student!
```

Макросы препроцессора с переменным числом параметров

Стандарт C99 допускает также и макросы с переменным числом параметров. Внутри макроса для ссылки на дополнительные параметры нужно использовать имя `__VA_ARGS__`.

В качестве примера напишем расширенный вариант макроса `assert`, который, помимо проверки, также выводит пользовательское сообщение:

```
#include <stdio.h>
#include <stdlib.h>

#define ASSERT_EX(cond, ... ) \
    if (! (cond)) \
    { \
        printf("File: %s, line %d, condition %s: ", \
               __FILE__, __LINE__, #cond); \
        printf(__VA_ARGS__); \
        exit(1); \
    }

int main()
{
    ASSERT_EX(2 * 2 == 4, "Ok");
    ASSERT_EX(2 * 2 == 5, "Hello to %d", 1984);
    return 0;
}
```

Для написания макроса с переменным количеством параметров, часть с переменным количеством нужно записать как эллипсис (`...`), для ссылки на неё нужно использовать `__VA_ARGS__`.

Этот пример также демонстрирует использование имён `__FILE__` и `__LINE__`, которые препроцессор заменяет на строковую константу с именем текущего файла и номером текущей строки. Если перед именем параметра макроса записать `#`, то на место параметра подставится строковая константа с текстом данного параметра.

Запуск программы даст ожидаемый результат: первая проверка выполнится успешно, вторая сообщит об ошибке:

```
D:\Си>a.exe
File: log-example.c, line 19, condition 2 * 2 == 5: Hello to 1984
```